

#{toc}

- [Objetivo desse documento](#)
- [Branches Oficiais](#)
- [Regras Gerais](#)
- [Passo a Passo: Desenvolvimento de Tarefa](#)
- [Passo a Passo: Subir para Homologação](#)
- [Passo a Passo: Tarefas homologadas](#)
- [Passo a Passo: Subir para Produção](#)
- [Limpeza](#)
- [Checklist para o Desenvolvedor](#)
- [Dicas úteis](#)
- [Resumo Visual](#)

Objetivo desse documento

Padronizar o uso do Git no time para evitar conflitos, garantir que apenas tarefas testadas sejam enviadas para produção, e manter a organização entre desenvolvimento, homologação e produção.

Branches Oficiais

Branch	Finalidade
main	Código que já foi homologado e está em produção
homol	Código que está em processo de homologação
devel	Código já homologado, que deve ser compartilhado entre a equipe

Regras Gerais

⚠ **NUNCA** trabalhar direto nos branches *main*, *devel* ou *homol*.

⚠ **NUNCA** faça merge de *homol* em nenhuma outra branch, independente do tipo, pois nesta branch pode existir código não aprovado.

ℹ O branch *homol* serve para acumular todas as tarefas que estão em processo de homologação. Ele **não deve ser usado como base de desenvolvimento** e pode ser reiniciado a qualquer momento.

Cada desenvolvedor deve trabalhar em sua própria branch de tarefa, sendo um branch isolado para cada tarefa.

Os merges devem ser manuais e controlados.

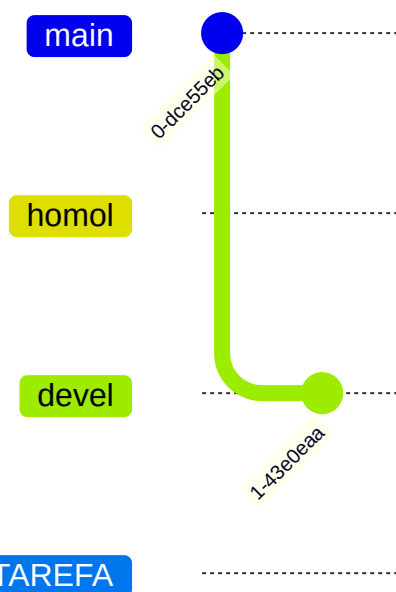
Evite conflitos mantendo seu repositório sempre atualizado.

Passo a Passo: Desenvolvimento de Tarefa

1. Criar uma branch de tarefa

i Sempre crie sua branch a partir do devel.

```
git checkout devel
git pull origin devel
git checkout -b feat/NOME-DA-TAREFA
```



Use nomes claros no padrão: feat/, fix/, chore/ + nome da tarefa, onde:

- fix: branch para correção de funcionalidade já existente
- feat: branch para desenvolvimento de nova funcionalidade ou processo
- chore: branch para documentação e refatoração de código

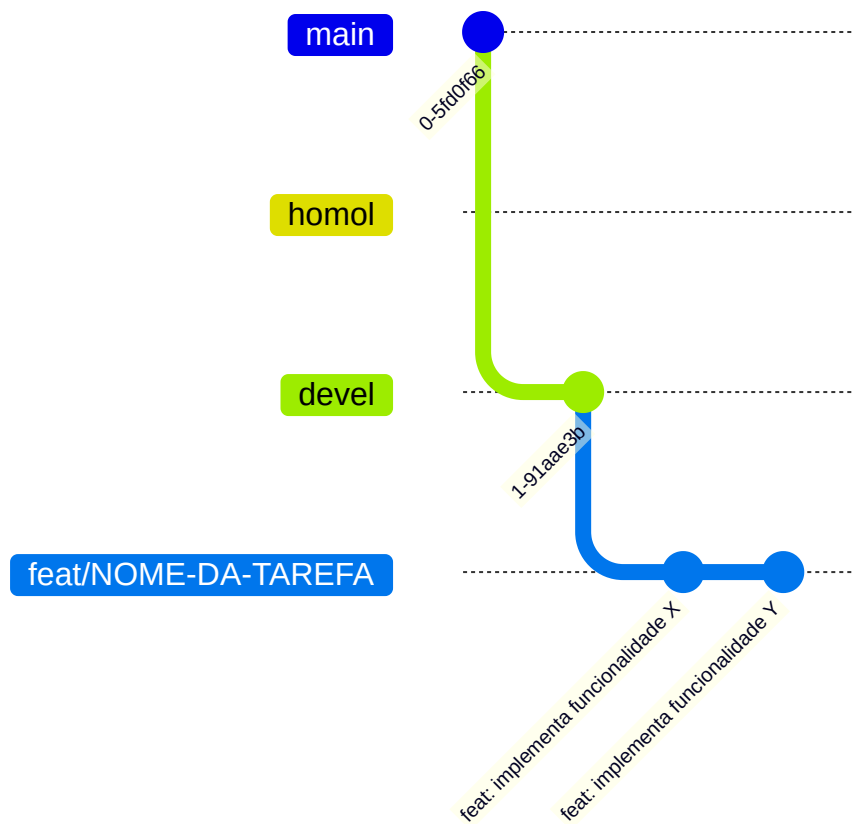
O uso de prefixos como feat/, fix/ e chore/ segue boas práticas amplamente adotadas, como as do [Conventional Commits](#), e favorece a automação de changelogs, releases etc.

Para manter-se um padrão o nome da tarefa deve ser o PPM do Jira.

2. Trabalhar na tarefa

Faça os commits normalmente:

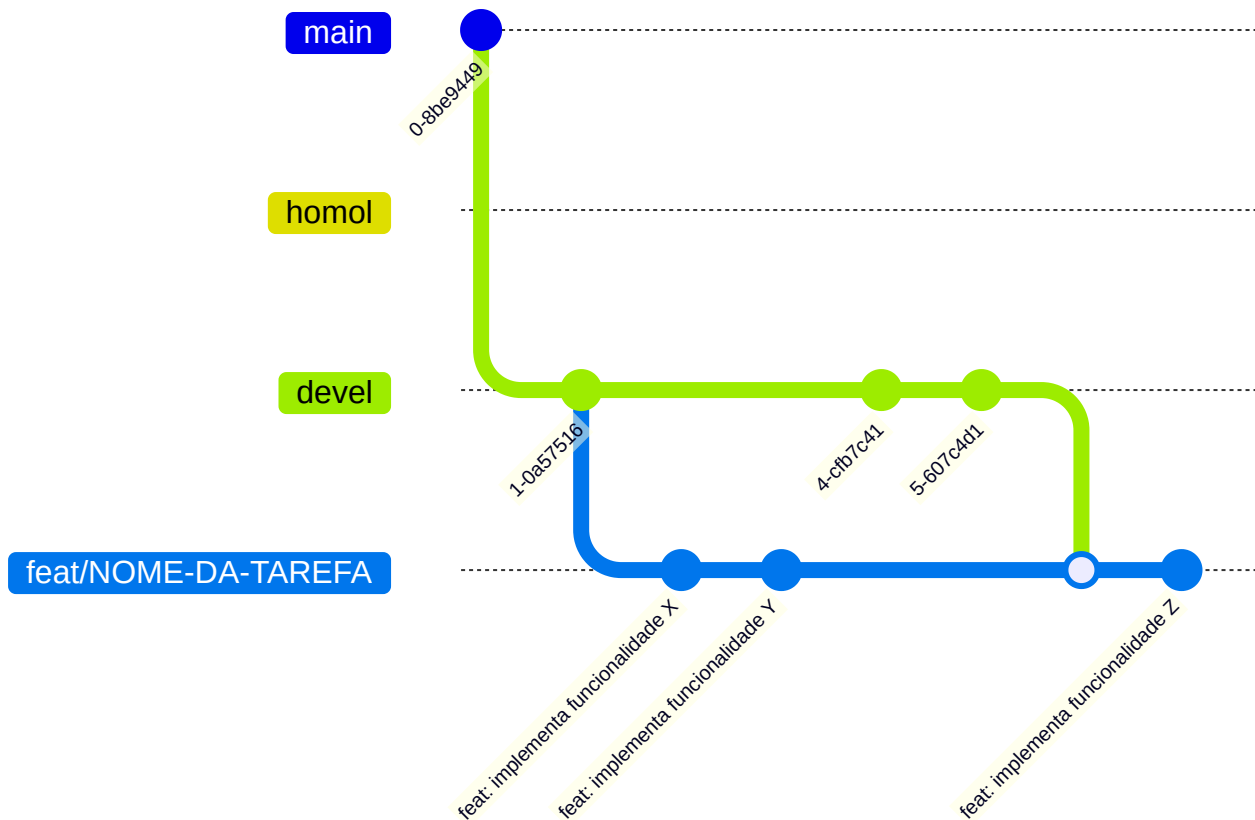
```
git add .
git commit -m "feat: implementa funcionalidade X"
```



Teste seu código localmente antes de prosseguir.

3. Atualizar sua branch com o devel (rotina diária), resolvendo imediatamente conflitos que possam ocorrer

```
git checkout devel
git pull origin devel
git checkout feat/NOME-DA-TAREFA
git merge devel
```



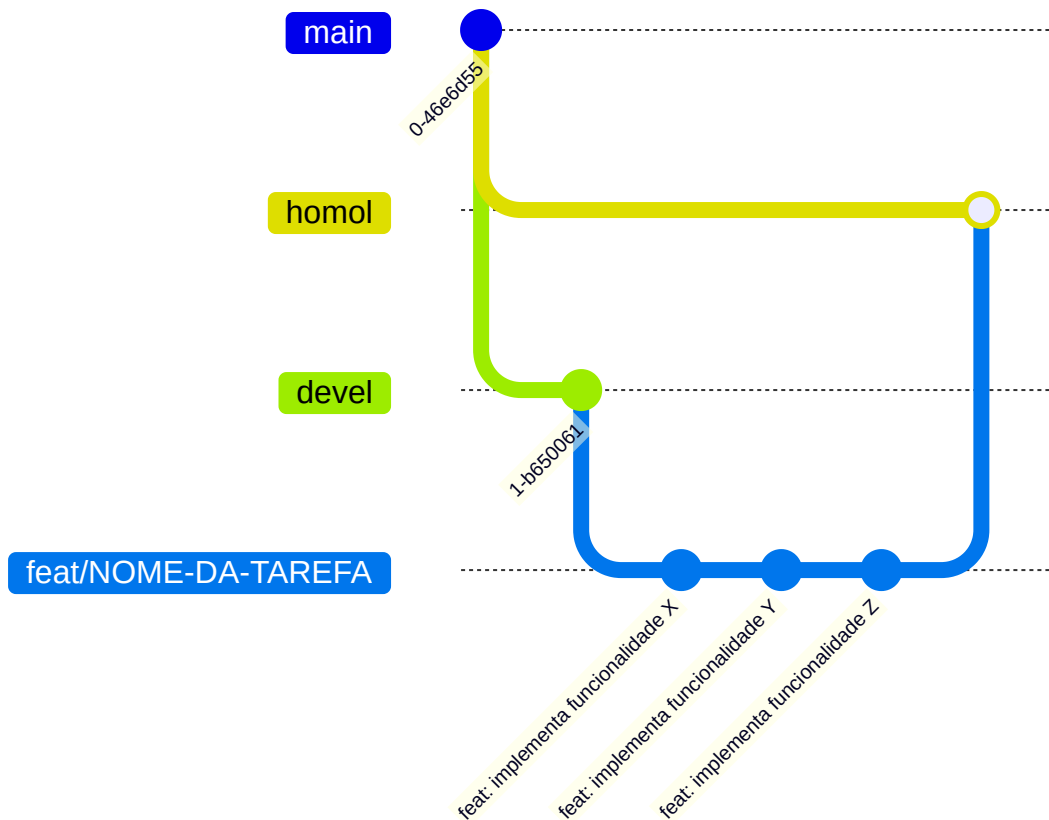
Passo a Passo: Subir para Homologação

Quando a tarefa ainda está em processo de desenvolvimento e homologação, deve-se garantir que os branches locais não contaminem o restante do projeto.

Para isso deve ser feito o merge do seu branch local para o *homol*, resolvendo imediatamente conflitos que possam ocorrer.

```
git checkout homol
git pull origin homol
git merge --no-ff feat/NOME-DA-TAREFA
git push origin homol
```

i O parâmetro `--no-ff` no merge força a criação de um commit de merge mesmo que o Git pudesse fazer um "fast-forward". Isso preserva o histórico da branch da tarefa no *homol*.



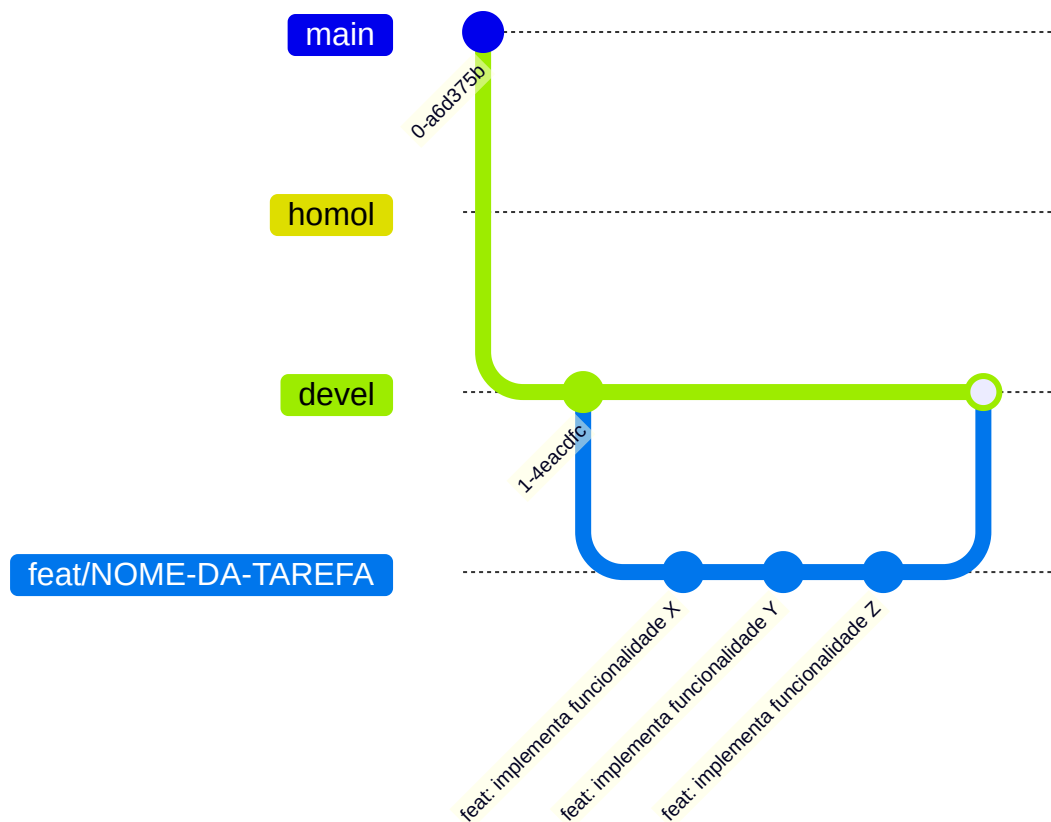
Passo a Passo: Tarefas homologadas

Quando o cliente homologar uma tarefa, deve ser feito o compartilhamento desse código com o restante da equipe.

Para isso deve-se fazer o merge do seu branch local para o *devel*, resolvendo imediatamente conflitos que possam ocorrer.

```
git checkout devel
git pull origin devel
git merge --no-ff feat/NOME-DA-TAREFA
git push origin devel
```

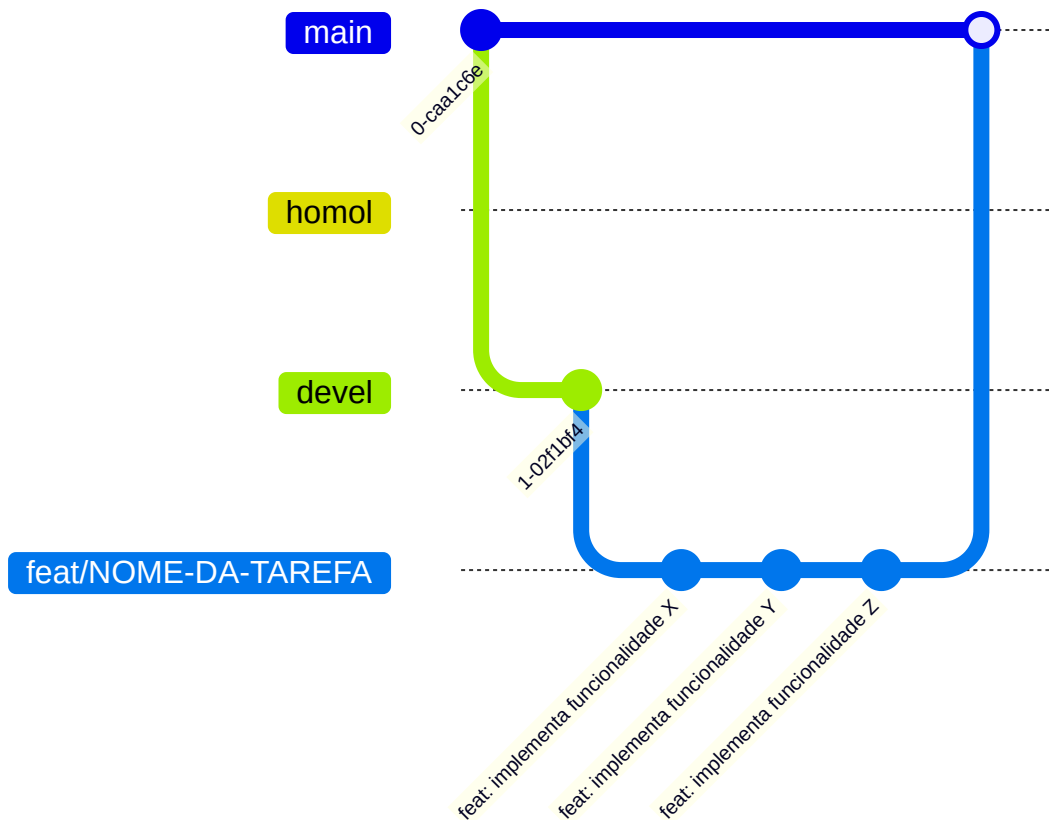
i O parâmetro `--no-ff` no merge força a criação de um commit de merge mesmo que o Git pudesse fazer um "fast-forward". Isso preserva o histórico da branch da tarefa no *homol*.



Passo a Passo: Subir para Produção

Quando o cliente autoriza a subida da tarefa para produção, o branch local deve ser enviado para *main*, resolvendo imediatamente conflitos que possam ocorrer.

```
git checkout main
git pull origin main
git merge feat/NOME-DA-TAREFA
git push origin main
```



Limpeza

Depois que sua tarefa foi publicada para a produção, o branch local se torna desnecessário, podendo ser eliminado.

```
git branch -d feat/NOME-DA-TAREFA      # Remove localmente
git push origin --delete feat/NOME-DA-TAREFA # Remove do remoto
git fetch --prune                        # Elimina referências aos
branches eliminados no remoto
```

O branch *homol* é efêmero e pode conter tarefas que foram iniciadas, mas abandonadas pelo cliente. Para evitar que o ambiente de homologação fique contaminado, recomenda-se que seja limpo de tempos em tempos

```
git checkout homol
git fetch origin
git reset --hard origin/devel
git push origin homol --force
```

Após isso os branches locais que ainda estão em desenvolvimento e em processo de homologação devem ser reaplicados ao branch *homol*.

Checklist para o Desenvolvedor

Antes de enviar para *homol*:

☐ A tarefa foi testada localmente?

Antes de enviar para *devel*:

- ☐ A tarefa foi homologada pelo cliente?
- ☐ Seu branch local está sincronizado com o devel?

Antes de enviar para *main*:

- ☐ O cliente aprovou o envio dessa tarefa para produção?

Dicas úteis

Use `git log --oneline` para ver commits rapidamente.

Use `git status` com frequência para saber o estado dos seus arquivos.

Use `git branch` para saber onde você está.

Em caso de dúvidas, pare e pergunte antes de fazer merge!

Resumo Visual

